КАК УСТРОЕН ПРОЕКТ

Расширенная учебная редакция: по файлам, слоям и блокам. Объяснение для новичка без знания C, с расшифровкой технических и англоязычных терминов.

Назначение документа	**Научить понимать устройство EH/Unwind Analyzer как инженерной системы: от C-файлов и сборки до PE-карты, unwind-декодера, эвристик, динамической проверки, отчётов, Ghidra-интеграции и тестов.**
Как читать	Не пытаться запоминать каждую строку. Нужно понять слои, структуры данных, движение данных между файлами и ответственность каждого модуля.
Ключевой тезис	Проект является не просто parser, а анализатором, который строит карту EH/Unwind-слоя PE-файла и превращает её в доказательные findings and reports.

1. Сначала общая картина: что делает проект

EH/Unwind Analyzer - это defensive analysis tool. Defensive означает, что инструмент предназначен для анализа и защиты, а не для обхода защит, эксплуатации уязвимостей или скрытого воздействия на чужие системы. Tool означает прикладной инструмент. В данном проекте инструмент работает с Windows PE32+ AMD64 файлами, то есть с 64-битными PE-файлами Windows для архитектуры x64.

Главная область проекта - EH/Unwind metadata. EH означает Exception Handling, то есть механизм обработки исключений. Unwind означает восстановление предыдущего контекста выполнения: как системе понять, где был вызывающий код, какие регистры надо восстановить и как пройти назад по стеку. Metadata означает служебные данные, которые описывают код, но сами не являются обычными инструкциями программы. В Windows x64 такие данные хранятся через Exception Directory, .pdata, RUNTIME_FUNCTION and .xdata/UNWIND_INFO.

Проект берёт бинарный файл, читает его как набор bytes, проверяет PE-структуру, находит таблицу функций для unwind, декодирует записи UNWIND_INFO, ищет структурные нарушения и подозрительные паттерны, а затем формирует отчёты. Если запуск происходит на Windows x64 и включён --dynamic, проект может дополнительно запустить target под отладчиком и собрать runtime evidence: исключения, попадания в handlers, регистрации VEH/VCH/UEF and dynamic function tables.

Новичку важно сразу отделить три уровня. Первый уровень - file format: как файл устроен на диске. Второй уровень - analyzer model: какие структуры проект строит внутри памяти. Третий уровень - report artifact: что получает человек в JSON, HTML, CSV или Ghidra CSV. Ошибка в понимании часто возникает, когда эти уровни смешивают. Например, RUNTIME_FUNCTION в файле - это bytes в Exception Directory; RuntimeFunction в коде проекта - это C-структура, в которую эти bytes прочитаны; строка в HTML - это уже представление этой структуры для человека.

```
[redacted]:

input file bytes
-> Buf: [redacted] bytes + size
-> PeImage: PE headers, sections, ImageBase, Exception Directory
-> RuntimeFunction[]: [redacted] .pdata
-> UnwindSummary[]: [redacted] UNWIND_INFO
-> HeuristicReport: findings, score, confidence, verdict
-> DynamicReport: runtime events, [redacted] --dynamic
-> JSON / HTML / CSV / Ghidra CSV / console output
```

2. Почему проект написан на C и что это меняет

C - низкоуровневый язык программирования. Low-level здесь означает, что язык даёт программисту прямой контроль над памятью, размерами типов, массивами bytes and системными API. Для проекта, который читает бинарные форматы, это естественный выбор: PE-формат состоит из полей точной длины, offset-ов, little-endian чисел, флагов и структур фиксированного размера.

Но C требует дисциплины. В C нет автоматической защиты от чтения за границы массива. Нет встроенного исключения, которое автоматически остановит программу при ошибке формата. Нет garbage collector, который сам освободит память. Поэтому в проекте много кода, который на первый взгляд кажется техническим: проверки размеров, функции buf_read, StrBuf, free, cleanup paths. Это не лишний шум. Это часть безопасности инструмента.

Если ты не знаешь C, читать проект нужно не как художественный текст сверху вниз, а как систему договоров. .h файл говорит: какие данные и функции существуют. .c файл показывает: как эти функции реализованы. CMakeLists.txt показывает: как .c файлы собираются в библиотеку and executable. main.c показывает: в каком порядке эти функции вызываются при реальном запуске.

Термин	Подробное объяснение
source code	Исходный код. Текст программы, который читает программист и компилирует compiler. В проекте это в основном файлы src/*.c and include/*.h.
compiler	Компилятор. Программа, которая переводит C-код в машинный код или объектные файлы. В проекте поддержаны GCC/Clang/MinGW/MSVC-подобные toolchains через CMake.
linker	Компоновщик. Он соединяет скомпилированные части в библиотеку или executable. Поэтому main.c может вызывать функции из pe.c, хотя они находятся в другом файле.
executable	Исполняемый файл. Итог сборки, который запускает пользователь: eh_unwind_analyzer or eh_unwind_analyzer.exe.
library	Библиотека. Набор скомпилированных функций без собственного main. В проекте ehua_core содержит ядро анализа, а CLI подключается к нему.

3. Мини-курс C именно для понимания проекта

3.1. .h и .c файлы

Header file, или .h файл, - это интерфейс. Interface означает внешний договор: какие структуры, константы и функции доступны другим частям программы. Например, include/pe.h объявляет PeImage and pe_parse. Другие файлы могут include-ить pe.h и знать, что такая функция существует.

Implementation file, или .c файл, - это реализация. В src/pe.c находится код, который реально проверяет MZ, PE signature, optional header, sections and RVA mapping. Поэтому нельзя говорить, что pe.h “делает анализ”. pe.h описывает контракт, а pe.c выполняет работу.

3.2. Функция, параметры и возвращаемое значение

Function - именованный блок кода. У функции есть return type, name and parameters. Return type говорит, что функция возвращает. Parameters - данные, которые ей передают. В проекте многие функции возвращают bool: true означает успех, false означает ошибку.

```
bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);

[illegible]:
- bool: [illegible] true/false;
- pe_parse: [illegible];
- PeImage* out: [illegible];
- const Buf* b: [illegible] bytes;
- StrBuf* err: [illegible].
```

3.3. struct

struct - структура данных, то есть набор полей под одним именем. В проекте почти все важные результаты представлены структурами. PeImage хранит карту PE-образа. UnwindSummary хранит итог разбора одного UNWIND_INFO. HeuristicReport хранит findings and score. DynamicReport хранит runtime events.

Структура полезна потому, что анализатор не должен передавать между файлами десятки отдельных переменных. Вместо этого pe.c заполняет PeImage, а остальные модули получают указатель на PeImage и читают нужные поля.

3.4. pointer, или указатель

Pointer, или указатель, - значение, которое хранит адрес другого объекта в памяти. В C это обозначается звёздочкой: PeImage* img. Если функция получает pointer, она может читать объект по этому адресу. Если объект не const, функция может его менять.

Out-parameter - параметр, через который функция возвращает дополнительный результат. Например, pe_parse возвращает bool как status, но сам PeImage результат записывает в PeImage* out. Это типичный стиль C: функция говорит “успешно/неуспешно” через return value, а подробный результат кладёт в структуру, адрес которой ей передали.

3.5. pointer to pointer

RuntimeFunction** out_rf выглядит страшно, но смысл простой. Одна звёздочка RuntimeFunction* означает “адрес массива RuntimeFunction”. Две звёздочки RuntimeFunction** означают “адрес переменной, в которую нужно записать адрес массива”. unwind_read_runtime_functions_ex сама выделяет массив через calloc, поэтому должна вернуть caller-у pointer на этот массив. Для этого нужен pointer to pointer.

```
RuntimeFunction* rf = NULL;
size_t rf_count = 0;
unwind_read_runtime_functions_ex(&b, &img, &rf, &rf_count, &err);

[illegible]:
- rf [illegible] RuntimeFunction;
- rf_count [illegible];
- main.c [illegible] free(rf).
```

3.6. fixed-size integer types

uint8_t, uint16_t, uint32_t and uint64_t - целые числа фиксированной ширины: 8, 16, 32 and 64 bits. Они нужны, потому что PE-формат задаёт размеры полей точно. Например, BeginAddress в RUNTIME_FUNCTION - 32-bit image-relative address. Если использовать тип неопределённого размера, парсер может работать по-разному на разных платформах.

size_t - тип для размеров памяти, индексов arrays and lengths. Он используется для file size, number of sections, rf_count, string length. В C это стандартный тип для величин, которые не должны быть отрицательными.

3.7. bitmask and flags

Bitmask - число, в котором отдельные bits означают отдельные признаки. Например, flags в UNWIND_INFO содержит bits UNW_FLAG_EHANDLER, UNW_FLAG_UHANDLER and UNW_FLAG_CHAININFO. Если bit установлен, соответствующий режим включён. В C это проверяют через оператор &: flags & UNW_FLAG_EHANDLER.

Anomalies в UnwindSummary тоже bitmask. Там могут одновременно стоять UA_TRUNCATED, UA_BAD_OPERAND, UA_CHAIN_LOOP. Это удобно: одна запись может иметь несколько проблем, и проект не теряет информацию.

3.8. heap memory, calloc, free and ownership

Heap memory - область памяти, из которой программа выделяет объекты во время выполнения. calloc выделяет memory and обнуляет её. free освобождает. Ownership означает, кто отвечает за освобождение. Если ownership не определён, в C быстро появляются memory leaks or double free errors.

В проекте ownership описан явно: buf_init_from_file выделяет input bytes, main.c освобождает через buf_free_owned. pe_parse выделяет PeImage.sections, main.c вызывает pe_free. unwind_read_runtime_functions_ex выделяет RuntimeFunction array, main.c вызывает free(rf). dynamic_report_init выделяет details buffer, main.c вызывает dynamic_report_free.

4. Архитектура: проект как конвейер слоёв

Архитектура проекта - modular pipeline. Modular означает, что система разделена на модули с отдельной ответственностью. Pipeline означает последовательную обработку: результат одного слоя становится входом следующего. Такой подход нужен, потому что EH/Unwind analysis нельзя надёжно делать одним куском кода. Сначала нужно доверенно читать bytes, потом понимать PE, потом понимать .pdata, потом понимать .xdata, потом оценивать аномалии, потом писать отчёт.

Если смешать всё в одном файле, ошибки будет трудно искать. Например, если JSON показывает неправильный section name, непонятно, ошибка в parsing, heuristics или report. В текущей архитектуре легче рассуждать: section map создаётся в pe.c; если section map правильна, report.c только отображает её; если finding wrong, смотреть heuristics.c; если parse_error wrong, смотреть unwind.c.

Слой	Файлы	Что значит простыми техническими словами
Foundation	util.c/.h, app_info.c/.h	Общие операции: безопасно читать bytes, строить строки, писать файлы, считать hash, знать версию продукта.
PE image model	pe.c/.h	Понять PE-контейнер: headers, sections, ImageBase, Exception Directory, RVA mapping.
EH/Unwind model	unwind.c/.h	Понять .pdata and .xdata: RUNTIME_FUNCTION, UNWIND_INFO, UWOP, handlers, chains, diagnostics.
Evidence model	heuristics.c/.h	Оценить, какие структурные признаки сработали and насколько они важны.
Runtime evidence	dynamic_win.c/.h	На Windows проверить runtime-события: exceptions, handler hits, VEH/VCH/UEF, dynamic function tables.
Output/integration	report.c/.h, ghidra/	Превратить модель в JSON/HTML/CSV/Ghidra CSV, чтобы человек and backend могли её использовать.
Validation	tests/, docs/schema, reports/	Проверять, что каждый слой ведёт себя ожидаемо and reports remain stable.

5. Физическая структура проекта на диске

Папки проекта отражают архитектуру. src/ and include/ - C-ядро. tests/ - проверка. docs/ - документация and report schema. ghidra/ - интеграция с внешней средой reverse engineering. scripts/ - вспомогательные операции. reports/ - evidence from validation. real_samples/ and “бинарники на тестирование” - corpus для проверки.

Важно понимать: build/ and release-output/ - производные папки. Они появляются после сборки или упаковки. Исходный смысл проекта находится не в build/, а в src/, include/, docs/, tests/ and ghidra/. Если передаёшь проект заказчику, эти папки показывают, что это не только executable, но и воспроизводимый source package.

Путь	Что внутри	Почему важно
src/	app_info.c, dynamic_win.c, heuristics.c, main.c, pe.c, report.c, unwind.c, util.c	Реальная реализация анализатора.
include/	app_info.h, dynamic.h, heuristics.h, pe.h, report.h, unwind.h, util.h	Публичные контракты между модулями.
CMakeLists.txt	Build graph, compiler options, tests, install rules	Показывает, как из исходников получается executable and как запускаются tests.
docs/report.schema.json	JSON Schema version 1.2.0	Формальный контракт отчёта для backend, tests and future frontend.
docs/evidence_matrix.md	Families, findings, score, confidence, weakening rules	Документирует, почему эвристики оцениваются именно так.
docs/windows_dynamic_validation.md	Checklist dynamic validation	Объясняет Windows-only runtime проверку.
tests/unit/	Unit tests C modules	Проверяют поведение core without UI.
tests/samples/	Synthetic/regression PE corpus generators and manifest	Даёт контролируемые test cases.
ghidra/import_unwind_csv.py	Ghidra importer	Переносит карту анализа в среду реверса.

6. CMakeLists.txt: как исходники становятся программой

CMakeLists.txt - это файл сборки. Build system - система, которая знает, какие файлы компилировать, какие options включить, какие tests зарегистрировать and что установить. В проекте CMake объявляет project eh_unwind_analyzer version 1.0.0, language C and standard C17.

Главное архитектурное решение в CMakeLists.txt - разделение core library and executable. Core library называется ehua_core. Она содержит app_info.c, dynamic_win.c, heuristics.c, pe.c, report.c, unwind.c and util.c. Executable называется eh_unwind_analyzer and создаётся из main.c, который линкуется с ehua_core. Link означает, что main.c получает доступ к функциям из библиотеки.

Почему это хорошо. Если core отдельно, его можно подключать к unit tests. Тесты могут вызывать функции pe_parse or unwind_parse напрямую, не притворяясь пользователем CLI. Это повышает проверяемость. Также future backend может в теории использовать core library напрямую, хотя практический и более простой путь - запускать CLI and читать JSON.

```
[illegible]:

src/app_info.c
src/dynamic_win.c
src/heuristics.c
src/pe.c
src/report.c
src/unwind.c
src/util.c
-> static library: ehua_core

src/main.c + ehua_core
-> executable: eh_unwind_analyzer
```

CMake also sets compiler warnings. Warning - предупреждение компилятора. Например, compiler может сказать: “здесь возможно неявное преобразование типа”, “эта переменная затеняет другую”, “формат printf не совпадает с типом аргумента”. В проекте warnings are strict, and EHUA_WARNINGS_AS_ERRORS makes warnings fatal. Это означает, что предупреждение может остановить сборку. Для системного C-инструмента это правильно: лучше исправить сомнительное место, чем получить скрытую ошибку на бинарном формате.

Sanitizers - инструменты проверки ошибок памяти and undefined behavior. Coverage instrumentation - режим, который показывает, какие строки кода покрыты тестами. Fuzzer - инструмент, который подаёт много мутированных входов и ищет crashes. Эти режимы не нужны обычному пользователю, но важны для качества продукта and защиты ВКР.

7. Главные структуры данных: скелет проекта

В C-проекте структуры данных важнее отдельных строчек. Если понять, какая структура что означает, дальше легче читать функции. EH/Unwind Analyzer строит несколько моделей. Модель - это внутреннее представление части реальности. PeImage - модель PE-файла. UnwindSummary - модель одного UNWIND_INFO. HeuristicReport - модель оценки. DynamicReport - модель runtime-сессии.

Структура	Где объявлена	Что означает подробно
Buf	util.h	Input bytes plus size. Это не “файл” в высокоуровневом смысле, а область памяти, где лежат bytes файла, и число, сколько bytes можно читать.
StrBuf	util.h	Mutable string buffer. Нужен для ошибок, details, JSON/HTML generation. Хранит pointer s, length len, capacity cap and failed flag.
PeSection	pe.h	Одна PE-секция: name, virtual_address, virtual_size, raw_ptr, raw_size, characteristics. Через неё проект понимает, где лежит .text/.pdata/.xdata.
PeImage	pe.h	Вся PE-карта: architecture, ImageBase, SizeOfImage, alignments, Exception Directory, sections and validation counters.
RuntimeFunction	unwind.h	Одна запись .pdata: begin_rva, end_rva, unwind_rva. Она связывает диапазон кода with UNWIND_INFO.
UnwindSummary	unwind.h	Декодированный UNWIND_INFO: version, flags, prolog size, count codes, handlers, chain, stack effects, anomalies and diagnostics.
HeuristicFinding	heuristics.h	Один структурированный признак: code, family, severity, message, location, affected_count.
HeuristicReport	heuristics.h	Полный результат эвристик: counters, findings, score, suspicion, confidence, verdict, mitigations.
DynamicEvent	dynamic.h	Одно runtime-событие: type, module, thread id, exception code, address, arguments.
DynamicReport	dynamic.h	Полный dynamic результат: status, counters, events, trace, instrumentation completeness, details.
ReportMetadata	report.h	Provenance отчёта: tool version, schema version, timestamp, SHA-256, command line, compiler, platform.

8. include/: почему заголовки - это карта договоров

Новичок часто начинает читать самый большой .c файл и быстро теряется. Правильнее начать с include/. Заголовки показывают, какие данные живут в проекте and как модули общаются. Это как формальная спецификация внутреннего API. API означает Application Programming Interface - интерфейс, через который один код пользуется другим кодом.

Каждый .h файл ограничивает область ответственности. util.h не знает про PE. pe.h не знает про score. unwind.h не пишет HTML. heuristics.h не запускает процесс. dynamic.h не парсит PE headers. report.h не говорит, как искать anomalies. Это разделение важно: оно предотвращает превращение проекта в один большой неразделимый файл.

Header	Что должен понять новичок
util.h	Здесь находятся самые базовые типы: Buf for bytes and StrBuf for growing text. Если понимаешь util.h, понимаешь foundation layer.
pe.h	Здесь PE представлен не как абстрактный файл, а как PeImage with sections and exception directory. Это начало карты.
unwind.h	Здесь описаны RUNTIME_FUNCTION and UNWIND_INFO summary. Это сердце предметной области.
heuristics.h	Здесь видно, что findings are structured data, not plain text. Это важно для JSON/backend/frontend.
dynamic.h	Здесь видно, какие runtime signals проект умеет хранить: exceptions, handler hits, registrations, dynamic tables.
report.h	Здесь видно, что output layer принимает все модели and metadata and writes reports.
app_info.h	Здесь закреплена version 1.0.0 and report schema version 1.2.0.

9. util.c / util.h: безопасное основание

util.c - фундамент проекта. Foundation означает нижний слой, на который опираются остальные. Этот файл не знает, что такое Exception Directory or UNWIND_INFO. Его задача проще and важнее: безопасно читать bytes, строить строки, писать файлы and считать SHA-256.

Почему это настолько важно. Анализатор читает недоверенный бинарник. Недоверенный означает, что файл может быть повреждён, специально сформирован против parser, иметь ложные sizes and offsets. Если parser будет читать bytes напрямую, он может выйти за границу input buffer. Поэтому все чтения проходят через buf_read and typed wrappers: buf_read_u8, buf_read_u16, buf_read_u32, buf_read_u64.

Typed wrapper означает функция-обёртка под конкретный тип. Например, buf_read_u32 читает 4 bytes and собирает uint32_t. Она не даёт остальному коду каждый раз вручную писать “проверь размер, скопируй bytes, собери little-endian value”. Это снижает риск ошибки.

```
■■■■■ safe byte read:

Input: Buf b, offset off, length len
Check 1: b exists
Check 2: off <= b.size
Check 3: len <= b.size - off
Only then: copy bytes
Otherwise: return false
```

StrBuf - второй базовый инструмент. В C строка обычно является массивом char, заканчивающимся нулевым байтом. Если писать в такой массив без контроля capacity, можно переполнить buffer. StrBuf хранит pointer на memory, текущую длину len, выделенную capacity cap and failed flag. Когда report.c строит HTML or JSON, он не должен угадывать заранее итоговый размер. StrBuf растёт по необходимости.

file_write_atomic - функция для безопасной записи output. Atomic в данном контексте означает: сначала пишется temporary sibling file, а потом он заменяет final file. Если запись сорвалась, старый отчёт не превращается в половину файла. Для report generator это профессиональная деталь.

sha256_hex считает SHA-256 hash input bytes. Hash - криптографический отпечаток файла. Он нужен не для “взлома”, а для воспроизводимости. Если отчёт содержит SHA-256, можно точно понять, какой файл анализировался, даже если имя файла изменилось.

> Как объяснять util.c: это не предметный анализатор, а слой безопасности чтения и сериализации. Если util.c ошибается в bounds checks, всё выше становится ненадёжным.

10. app_info.c / app_info.h: версия, schema and provenance

app_info.c маленький, но продуктово важный. Он отвечает за version and provenance. Version - версия программы. Schema version - версия формата JSON report. Provenance - происхождение артефакта: чем, когда and в какой build configuration он создан.

Функции app_version and app_report_schema_version возвращают строки “1.0.0” and “1.2.0”. Функции app_build_compiler, app_build_platform and app_build_mode позволяют отчёту показать compiler, platform and debug/release mode. app_utc_timestamp формирует timestamp in UTC in RFC 3339 format. UTC timestamp важен, потому что отчёт может читать человек в другой timezone or backend on server.

Без app_info.c отчёт был бы менее воспроизводимым. Представь, что через полгода изменились thresholds, schema fields or HTML layout. Если в старом JSON нет tool_version and schema_version, трудно понять, как его интерпретировать. Поэтому app_info.c - часть report trustworthiness.

11. pe.c / pe.h: построение PE image map

pe.c - первый предметный модуль. Он превращает Buf into PeImage. PE означает Portable Executable - формат исполняемых файлов Windows. Portable здесь исторический термин: формат используется для разных типов executable images. PE32+ означает 64-bit variant. AMD64 означает x64 architecture.

pe_parse начинает с DOS header. DOS header содержит magic MZ and поле e_lfanew. e_lfanew указывает offset, где начинается PE header. Затем pe.c проверяет PE signature. Signature - фиксированная метка формата. Если signature wrong, дальше читать нельзя: это может быть не PE-файл or повреждённый input.

Дальше pe.c читает COFF header. COFF header содержит Machine and NumberOfSections. Machine должен быть IMAGE_FILE_MACHINE_AMD64. Это важно: x86, ARM64 or other architectures имеют другие правила unwind. Проект честно ограничен Windows x64.

Optional Header в PE на самом деле не optional для executable image. В нём лежат ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders and DataDirectory. ImageBase - preferred base address образа в памяти. SectionAlignment - выравнивание секций in virtual memory. FileAlignment - выравнивание секций in file. SizeOfImage - размер image in memory. SizeOfHeaders - область headers. DataDirectory - таблица каталогов, где Exception Directory имеет индекс 3.

PE term	Подробное объяснение
RVA	Relative Virtual Address. Адрес относительно ImageBase внутри загруженного образа. RUNTIME_FUNCTION uses RVA, not file offset.
VA	Virtual Address. Полный адрес в памяти: ImageBase + RVA. В static layer обычно VA не нужен, но dynamic layer видит runtime addresses.
file offset	Положение bytes внутри файла на диске. Чтобы прочитать UNWIND_INFO, нужно перевести RVA to file offset.
section	Диапазон файла/образа с именем, virtual address, raw pointer and permissions. .text обычно code, .pdata runtime functions, .xdata unwind info.
characteristics	Битовые flags секции: readable, writable, executable and others. Они нужны для placement heuristics.
Exception Directory	DataDirectory entry, который указывает на function table used for exception handling/unwind on x64.

pe.c также считает structural counters: section_raw_oob, section_virtual_oob, section_raw_overlaps, section_virtual_overlaps, section_misaligned, exception_directory_oob. Counter - счётчик. Он показывает, сколько раз найдено нарушение определённого типа. Эти counters потом попадают в heuristics.c and report.c.

Функция pe_find_section_for_rva ищет секцию, которая покрывает given RVA. Функция pe_rva_to_fileoff делает следующий шаг: если RVA лежит в секции and попадает в raw bytes, она вычисляет physical file offset. Если RVA попадает в virtual tail, который exists in memory but not in file, static parser cannot read it from disk and returns false. Это важное различие: не каждый valid memory RVA имеет bytes in file.

```
RVA mapping rule in project:

Find section where:
section.VirtualAddress <= RVA < section.VirtualAddress + max(VirtualSize, RawSize)
Then:
delta = RVA - section.VirtualAddress
if delta >= RawSize: not mappable from file
file_offset = section.RawPointer + delta
```

Как защищать pe.c: он строит систему координат. Если координаты wrong, unwind.c будет читать не те bytes. Поэтому PE layer - не формальность, а основание всей карты.

12. unwind.c / unwind.h: декодер .pdata and .xdata

unwind.c - предметное ядро проекта. Он занимается Windows x64 exception/unwind structures. Сначала он читает Exception Directory as RUNTIME_FUNCTION table. На x64 каждая RUNTIME_FUNCTION entry содержит три 32-bit RVA: BeginAddress, EndAddress and UnwindInfoAddress. BeginAddress and EndAddress задают half-open range [Begin, End). Half-open значит: начало включено, конец не включён. Это удобно для interval lookup.

unwind_read_runtime_functions_ex проверяет Exception Directory carefully. Если RVA/Size both zero, table отсутствует. Если один zero, а другой non-zero, это malformed. Если directory RVA не переводится to file offset, malformed. Если directory size выходит за file range, malformed. Если size not multiple of 12, malformed, потому что каждая RUNTIME_FUNCTION запись занимает 12 bytes.

После чтения RuntimeFunction[] main.c вызывает unwind_parse for each entry. unwind_parse читает UNWIND_INFO by UnwindInfoAddress. UNWIND_INFO starts with 4-byte header: Version/Flags, SizeOfProlog, CountOfCodes, FrameRegister/FrameOffset. Затем идёт массив UNWIND_CODE slots. Slot - двухбайтовая единица. Важно: CountOfCodes считает slots, not human operations. Одна UWOP operation может занимать больше одного slot.

UNWIND term	Что это значит
Version	Версия формата UNWIND_INFO. Проект поддерживает version 1 and 2. Unknown version becomes anomaly.
Flags	Биты, которые говорят, есть ли EHANDLER, UHANDLER or CHAININFO tail.
SizeOfProlog	Размер пролога функции в bytes. CodeOffset operations должны относиться к прологу.
CountOfCodes	Количество UNWIND_CODE slots, а не количество логических операций. Ошибка здесь сдвигает чтение tail.
UWOP	Unwind operation. Код операции, который описывает push/save/alloc/frame/epilog/machframe effect.
padding	Выравнивающий пустой slot, если CountOfCodes нечётный. Tail начинается после even number of slots.
handler RVA	RVA обработчика исключений/unwind, если flags contains EHANDLER or UHANDLER.

UNWIND term	Что это значит
CHAININFO	Режим, где tail содержит chained RUNTIME_FUNCTION-like record, а не handler RVA.

slots_for_code is a key helper. It decides how many slots a UWOP occupies. For example, UWOP_PUSH_NONVOL uses one slot. UWOP_ALLOC_LARGE can use two or three slots depending on opinfo. SAVE_NONVOL and SAVE_XMM operations have operand slots. If parser assumed “one operation equals one slot”, it would read tail from wrong offset. This project does not make that mistake: it advances i by slots and checks i + slots <= CountOfCodes.

Padding is handled by slots_even = (CountOfCodes + 1) & ~1. This expression rounds CountOfCodes up to even number. If CountOfCodes is odd, one padding slot is included before tail. This is crucial for handler RVA and CHAININFO. Without it, parser would read two bytes too early and interpret padding as part of handler RVA.

unwind.c also records estimated fixed_stack_bytes, pushed_nonvol_count, saved_nonvol_mask and saved_xmm_mask. Это не полное доказательство соответствия прологу. Это декодированная модель того, что metadata says. To fully prove consistency, future improvement would disassemble .text and compare UWOP effects with actual prolog instructions. Current project deliberately focuses on structural metadata analysis.

1. Take UnwindInfoAddress from RUNTIME_FUNCTION.
2. Remember that it is RVA, not file offset.
3. Map RVA to file offset through pe_rva_to_fileoff.
4. Read UNWIND_INFO header.
5. Split Version and Flags from first byte.
6. Validate supported version and flags combination.
7. Compute slots_even and verify available bytes for UNWIND_CODE array.
8. Parse each UWOP with correct extra-slot accounting.
9. Record anomalies: bad opcode, bad operand, code offset outside prolog, bad ordering, strict ABI warnings.
10. Read tail according to flags: handler RVA or chained record.
11. Optionally follow chain with max depth and loop detection.
12. Store diagnostic field, offsets, expected/available sizes and bytes when parsing fails.

13. heuristics.c / heuristics.h: evidence engine

heuristics.c - это evidence engine. Evidence means доказательное наблюдение. Engine means module that applies rules systematically. Этот файл не читает raw PE bytes. Он получает готовые models: PeImage, RuntimeFunction[], UnwindSummary[] and us_ok[]. Его задача - определить, какие признаки сработали.

Важно: heuristics.c не является neural network, statistical model or malware classifier. Neural network - обучаемая модель, которая делает вывод на основе training data. Statistical model - модель с вероятностной калибровкой. Malware classifier - классификатор, который говорит “malware/benign”. В проекте этого нет. Здесь deterministic rules: фиксированные правила, weights and thresholds described in docs/evidence_matrix.md.

Finding - центральное слово. Finding means structured detected signal. В проекте finding имеет code, family, severity, message, optional representative_rva and affected_count. Code - машинно-читаемое имя. Family - семейство evidence. Severity - серьёзность наблюдения. Message - краткое объяснение. representative_rva - адрес, с которого можно начать ручную проверку. affected_count - сколько объектов затронуто.

Family	Расшифровка
table_integrity	Целостность таблиц PE/.pdata: ranges, sizes, sorting, overlaps, duplicate begins, Exception Directory bounds.
unwind_integrity	Корректность .xdata/UNWIND_INFO: parse failures, opcode/operand anomalies, chain problems, ABI warnings.
metadata_placement	Размещение metadata and handlers in sections: normal, unusual, executable, writable, non-executable.
unwind_reuse	Повторное использование одного UnwindInfoRVA many times. Может быть benign, но with context может signal dispatcher/protector pattern.
handler_dominance	Dominant handler target. Один handler обслуживает many functions, что требует context and dynamic validation.

heuristics_analyze starts by copying PE validation counters. Then it loops through every RuntimeFunction entry. It checks invalid ranges, out-of-image ranges, missing or out-of-bounds unwind RVA, sorting inversions, parse failures and anomalies. It also counts reuse by storing occurrences of each UnwindInfoRVA and each HandlerRVA in internal hash maps.

Hash map here means data structure for counting keys efficiently. Key can be RVA, value can be count. Without hash maps, reuse analysis would require nested loops and become slow on large tables. With hash maps, project can compute max count and unique count more directly.

Score is suspiciousness score from 0 to 100. Suspicion is score normalized to 0..1. Confidence is confidence in interpretation, not probability of maliciousness. Severity and confidence are different. Severity answers: how serious is this finding if true? Confidence answers: how much do we trust this interpretation based on evidence context? A high-confidence low-severity finding is possible. A potentially high-severity but low-confidence hypothesis is also possible.

Weakening rules reduce confidence when evidence is too isolated. single_signal_only means only one non-strong finding exists. small_sample_size means there are too few functions for stable ratios. placement_only_signal means only unusual placement is present without stronger corroboration. single_family_only means evidence comes from one family only. This prevents overclaiming.

> Защитная фраза: heuristics.c не “угадывает вредоносность”, а систематически превращает структурные факты EH/Unwind в проверяемые findings with severity, confidence and manual review policy.

14. dynamic_win.c / dynamic.h: runtime evidence collector

dynamic_win.c - Windows-only dynamic layer. Windows-only means: full logic works only on Windows. On macOS/Linux the file compiles a stub implementation that says dynamic mode is supported only on Windows. This is correct because module depends on Windows Debug API.

Runtime means “во время выполнения”. Static layer reads file on disk. Dynamic layer observes process execution. Process means running program instance. Debug API means official Windows interface for debugger-like observation: CreateProcessA starts target under debug mode, WaitForDebugEvent receives events, ContinueDebugEvent lets target continue. Events include process creation, DLL load/unload, exceptions and process exit.

The dynamic module uses software breakpoints. Breakpoint means intentional stop point inserted into target execution. Software breakpoint on x86/x64 usually writes INT3 byte. The tool saves original byte, writes INT3, catches EXCEPTION_BREAKPOINT, restores original byte, adjusts RIP, single-steps, then re-arms breakpoint. This is why dynamic_win.c has many helper structures: Breakpoint, BpList, PendingRearmList.

Runtime signal	Что означает
exception	Target generated an exception. Report stores exception code, address, thread id and first/second chance state.
unwind_handler_hit	Execution reached handler RVA that static UNWIND_INFO referenced. This correlates static metadata with runtime path.
veh_registration	Process called AddVectoredExceptionHandler. VEH means vectored exception handler.
vch_registration	Process called AddVectoredContinueHandler. VCH means vectored continue handler.
uef_registration	Process called SetUnhandledExceptionFilter. UEF is top-level unhandled exception filter.
rtl_add_function_table	Process called RtlAddFunctionTable, registering runtime function table for generated/dynamic code.
rtl_install_function_table_callback	Process registered callback-based dynamic function table. Static .pdata is not the full runtime map.
trace	Limited sequence of RIP addresses collected after handler hit. RIP is instruction pointer register on x64.

DynamicReport stores both events and quality counters. instrumentation_complete tells whether collector believes instrumentation was complete. module_refresh_failures, breakpoint_set_failures, pending_rearm_failures, dropped_events and timed_out show limitations. This is essential: a dynamic run can collect useful evidence even if incomplete, but report must not overstate absence of events.

Dynamic validation is not sandboxing. Sandbox means controlled environment for executing unknown code. This project can run target under debugger, but it does not itself provide full isolation. For malware or unknown samples, dynamic mode must be executed inside isolated Windows x64 VM, ideally with snapshots, no shared credentials and controlled network. Static mode is passive and cross-platform; dynamic mode crosses execution boundary.

> Комиссии надо говорить честно: dynamic confirms observed behavior in one run. It does not prove that unobserved behavior cannot occur on another path, input, environment or anti-debug condition.

15. report.c / report.h: отчёт как часть анализа

report.c - самый большой source file, because output layer is rich. Новичок может подумать: “отчёты - это просто печать”. В этом проекте это не так. Report layer превращает internal models into audit artifact. Audit artifact означает результат, который можно проверить, сохранить, передать, сравнить and использовать в другом software.

JSON report - machine-readable contract. Machine-readable означает, что его удобно читать программам. Backend can parse JSON, validate schema, store fields in database and serve frontend. docs/report.schema.json defines what fields must exist and what types are allowed. Schema version protects compatibility.

HTML report - human-readable dashboard. Human-readable означает удобный для человека. HTML содержит summary cards, sections, matrix of 27 heuristic signals, static coverage panel, findings with Russian explanations, diagnostic errors, dynamic evidence table and manual validation steps. It is self-contained and offline, with Content Security Policy, no remote scripts or fonts.

CSV report - flat table. Flat означает табличный формат without nested objects. It is convenient for spreadsheets and quick review. Ghidra CSV is specialized integration output for import_unwind_csv.py.

Report feature	Why it matters
report_metadata	Shows tool_version, schema_version, timestamp, SHA-256, command line, compiler/platform/build mode.
pe_validation	Preserves PE layout counters so backend/frontend can show structural state.
functions array	Each RuntimeFunction with decoded unwind summary. This is the main static map.
heuristic_analysis	Findings, score, confidence, evidence families, mitigations, manual review state.
dynamic_validation	Runtime events and session quality counters when --dynamic used.
assessment	Final combined static/dynamic case strength.
evidence chain	Shows path PE -> Exception Directory -> RuntimeFunction -> UnwindInfo -> diagnostic/finding.

report.c also performs escaping. Escaping means converting unsafe characters into safe representation for JSON or HTML. If input path or section name contains quotes, angle brackets or invalid UTF-8, report writer must not create broken JSON or HTML injection. This is why report.c contains json_escape, html_escape and UTF-8 handling.

The HTML finding explanation functions are pedagogically important. finding_title_ru gives short Russian title. finding_meaning_ru explains technical meaning. finding_rule_text states normative rule. finding_expected_text states expected model. finding_impact_text explains impact. finding_next_step_ru tells manual validation plan. This makes report more than a list of codes.

> Если на защите спросят, где project explains its output, отвечай: в report.c output layer формирует evidence chain and Russian explanations, а docs/report.schema.json freezes machine-readable contract.

16. main.c: orchestrator, not analyzer brain

main.c contains main function - entry point. Entry point means the first function called when executable starts. main.c parses command line options, controls lifecycle and calls modules in order. It is orchestration layer. Orchestration means coordination: main.c does not itself know all details of PE and unwind, but it knows which module to call and when.

parse_args reads CLI options. CLI means command-line interface. Arguments include input path, --dump, --limit, --ops, --heuristics, --dynamic, --json, --html, --csv, --ghidra-csv, --strict-abi, --no-follow-chain, --max-chain-depth, --timeout-ms, --trace-steps and target arguments after --. Target arguments are arguments passed to analyzed executable in dynamic mode.

After parsing arguments, main.c reads file into Buf, initializes StrBuf err, calls pe_parse, reads RuntimeFunction array, allocates UnwindSummary array and us_ok array, calls unwind_parse in loop, optionally prints dump, optionally runs heuristics, optionally runs dynamic collector, prepares metadata for reports, writes outputs, frees memory and returns exit code.

1. Parse command line and handle --help/--version.
2. Read input file into memory through util.c.
3. Build PeImage through pe.c.
4. Read RuntimeFunction[] through unwind.c.
5. Decode every UNWIND_INFO into UnwindSummary[].
6. If requested, print dump and operations.
7. If requested, create HeuristicReport.
8. If requested, create DynamicReport on Windows.
9. Prepare ReportMetadata: timestamp, SHA-256, command line, version and build info.
10. Write CSV, Ghidra CSV, JSON and HTML as requested.
11. Free all owned resources in reverse-ish lifecycle order.
12. Return exit code: 0 success, 1 analysis/output/runtime error, 2 invalid command line.

The cleanup part is not optional. In C, every allocated memory block needs release. main.c handles early failures carefully. For example, if pe_parse fails, it frees err and input bytes before returning. If allocation of UnwindSummary fails, it frees runtime functions, PeImage and input. This is part of robust engineering.

17. Движение данных: полный путь одного файла

Теперь свяжем всё в один путь. Представим, что пользователь запускает command: eh_unwind_analyzer target.exe --heuristics --json report.json --html report.html. Сначала operating system creates analyzer process. main receives argc/argv. parse_args turns raw strings into Args structure. Args is internal CLI configuration.

Then buf_init_from_file opens target.exe, reads bytes and fills Buf. Buf is passed by pointer to pe_parse. pe_parse does not own Buf; it borrows it. Borrow means function uses object but does not free it. pe_parse fills PeImage and allocates PeImage.sections. Now analyzer knows ImageBase, sections and Exception Directory.

Next unwind_read_runtime_functions_ex uses PeImage.exception_rva and exception_size. It maps exception_rva to file offset using pe_rva_to_fileoff and reads triples into RuntimeFunction array. The caller owns this array. If directory is malformed, project can mark

exception_directory_oob and continue partial report.

Then main allocates UnwindSummary array with one element per RuntimeFunction and bool us_ok array. For each RuntimeFunction, it calls unwind_parse. Even if one UNWIND_INFO fails, project keeps parse_error and diagnostic inside corresponding UnwindSummary. This allows report to say “entry 77 failed at CountOfCodes / UWOP operand slots”.

When static models are ready, heuristics_analyze computes findings. It never mutates PeImage or UnwindSummary; it reads them and creates HeuristicReport. If --dynamic is enabled, dynamic_collect_windows uses static models as guide: handler RVAs from UnwindSummary become breakpoint targets. Finally report.c serializes all models into outputs.

```
Object ownership in one run:

main owns input bytes -> buf_free_owned
pe_parse owns sections -> pe_free
unwind reader owns rf array -> free(rf)
main owns us/us_ok arrays -> free(us), free(us_ok)
heuristics owns details -> heuristics_free
dynamic owns details -> dynamic_report_free
report writers own temp sb -> local sb_free inside report.c
```

18. Static layer: что именно считается статикой

Static analysis means анализ без выполнения target. The analyzer opens PE file as bytes and reasons about its declared structure. Static is safer and cross-platform. In this project static layer works on macOS/Linux/Windows, because it does not require Windows APIs for execution.

Static layer answers: what map does the file declare? It does not answer: what branch will execute, whether handler will be hit at runtime, whether target will call RtlAddFunctionTable during execution. Those are dynamic questions. Static layer can only observe file-backed structures and clues.

Static layer consists of PE validation, RuntimeFunction reading, UNWIND_INFO decoding, placement/reuse/dominance analysis and reports. It catches malformed structures, suspicious section placements, invalid or overlapping ranges, parse failures, chain loops, handler OOB and suspicious combinations. It also records when no strong anomalies are found.

Static sublayer	Questions it answers
PE validation	Is this PE32+ AMD64? Are headers/sections mappable? Is Exception Directory inside image?
RuntimeFunction table	How many entries? Are they readable, sorted, non-overlapping, inside SizeOfImage?
UNWIND_INFO parsing	Can header, codes, padding, handler tail or chain tail be decoded safely?
Metadata placement	Where do unwind metadata and handlers live? Is section executable, writable, unusual, non-executable?
Reuse/dominance	Do too many functions point to one metadata record or one handler?
Report evidence	Can every finding be traced back to field, RVA, section, count and manual validation step?

19. Dynamic layer: что он добавляет к статике

Dynamic analysis means observation while target is running. In this project dynamic is validation, not replacement of static. It takes static hypotheses and tries to collect runtime facts. The important word is facts: exception event occurred, handler breakpoint was hit, VEH registered, RtlAddFunctionTable called, process timed out. Dynamic report should not invent facts that were not observed.

Why dynamic matters. Windows x64 unwind information is not limited to .pdata on disk. Runtime can register dynamic function tables. JIT compiler, language runtime, profiler, protector or instrumentation framework can create code in memory and register unwind metadata for it. Static file map cannot see a table that is created only at runtime. Dynamic mode can observe API calls that create such tables.

Dynamic also checks exception routes. Static may say: this function has handler RVA. Dynamic can show whether that handler got control in this run. If yes, static handler route has runtime corroboration. If no, it does not mean route impossible; maybe input did not trigger it. Therefore dynamic result must always be read with timeout, coverage and instrumentation counters.

20. Что именно получает реверс-инженер

Reverse engineer receives not source code internals, but report artifacts. The most important outputs are JSON and HTML. JSON is for machines and integration. HTML is for human review. CSV/Ghidra CSV are auxiliary but useful.

The report tells the analyst: what file was analyzed, what its hash and size are, which tool version produced report, what PE sections exist, where Exception Directory points, how many RuntimeFunction entries were parsed, which UNWIND_INFO records are valid, what handlers/chains exist, which heuristic signals fired, what score/confidence/verdict are, what dynamic events were observed and what manual validation steps remain.

This is why project is not “just parser”. Parser output would list fields. Analyzer report creates navigable map. It says: start at this finding, go to this RVA, see this function range, inspect this UNWIND_INFO, note this section permissions, verify this manually in Ghidra or debugger.

Report field/block	Interpretation for beginner
input	Path displayed for analyzed target. It identifies what user asked to analyze.
report_metadata	Provenance: version, schema, timestamp, SHA-256, input size, command line, compiler/platform.
sections	PE section map with virtual/raw ranges and readable/writable/executable flags.
functions	RuntimeFunction records with begin/end/unwind RVA and decoded unwind summary.
heuristic_analysis.findings	Structured anomaly signals. Each finding has code, family, severity and evidence location.
dynamic_validation.events	Runtime evidence collected under Windows debugger.
assessment	Combined summary of static/dynamic evidence level and case strength.

21. Ghidra integration: зачем она нужна

Ghidra is reverse engineering platform. It can disassemble, decompile and annotate binary code. EH/Unwind Analyzer does not replace Ghidra. It generates specialized EH/Unwind evidence and exports it as Ghidra CSV. The script ghidra/import_unwind_csv.py imports function ranges and comments/bookmarks into Ghidra.

Why not do everything inside Ghidra? Because analyzer is standalone, batch-friendly and produces JSON/HTML/CSV independent of GUI. Ghidra integration is optional workflow bridge. Bridge means connector between automated structural analysis and manual reverse engineering environment.

On defense, this is a strong architecture: core analyzer remains CLI and automatable; Ghidra remains expert workspace; integration layer transfers map from one to the other without merging responsibilities.

22. Tests: как проект доказывает, что не разваливается

Testing in this project is multi-layered. Multi-layered means that tests check different responsibilities, not just “program starts”. Unit tests check modules. Regression samples check end-to-end behavior on known PE files. JSON schema validates report format. Semantic validators check cross-field consistency. Dynamic smoke checks Windows runtime collector. Fuzzing checks robustness against mutated inputs.

Unit test means small automated test of a unit, such as a function or module. Regression test means test that prevents old bug from returning. Smoke test means quick check that a large subsystem basically works. Fuzzing means feeding many malformed inputs to find crashes. Schema validation means checking JSON against formal schema. Semantic validation means checking that fields make sense together, not just have correct types.

Test category	What it protects
unit_unwind	UNWIND_INFO decoding rules, slots, flags, anomalies.
unit_heuristics*	Finding logic, mitigations, context and combinations.
unit_report_json/html	Output escaping, schema-friendly fields, HTML content.
regression_samples	Known PE samples produce expected outputs.
dynamic_validator	Dynamic report structure and evidence rules.
report_semantics	JSON report fields are internally consistent.
mutation_fuzz	Parser resists mutated bytes without crashes.
release_hygiene/metadata	Release package stays clean and versioned.

Current project evidence includes 16 CTest gates in normal run, Windows dynamic smoke scenarios for VEH, VCH, UEF and runtime function tables, and real corpus validation. But the most important explanation is not only number of tests. The important point is coverage by responsibility: bytes, PE, unwind, heuristics, report, dynamic, release. This matches architecture.

23. Пофайловый разбор: что отвечать по каждому файлу

Файл	Как объяснять подробно
src/main.c	CLI and orchestration. It parses options, calls core modules, manages lifecycle and cleanup, writes selected outputs, returns exit code. It should not contain deep PE logic.
src/util.c	Foundation for safe input and output: bounded byte reads, little-endian integer reads, StrBuf, atomic file writing, SHA-256. Every parser module relies on it.
src/pe.c	PE image model builder. Validates MZ/PE/PE32+/AMD64 headers, sections, alignments, Exception Directory, and provides RVA-to-file-offset mapping.
src/unwind.c	EH/Unwind decoder. Reads RUNTIME_FUNCTION table, parses UNWIND_INFO, accounts UWOP slots and padding, handlers, CHAININFO, anomalies and diagnostics.
src/heuristics.c	Evidence engine. Counts structural issues, placement, reuse and handler dominance; creates findings, score, confidence, verdict and mitigations.
src/dynamic_win.c	Windows runtime collector. Uses Debug API, breakpoints and context handling to observe exceptions, handlers and dynamic function-table API calls.
src/report.c	Output layer. Serializes models to console/CSV/JSON/HTML/Ghidra CSV and explains findings in Russian with evidence chains.
src/app_info.c	Product metadata. Provides version, schema version, compiler/platform/build mode and UTC timestamp for reports.
include/*.h	Contracts and data model. They define what each module exposes and what structures move through pipeline.
CMakeLists.txt	Build and test definition. Shows core library, executable, warnings, sanitizers, coverage, fuzzer and CTest registration.

24. Как новичку читать исходники без паники

Do not start with report.c. It is large because it prints many formats and explanations. Start with headers and the pipeline. The goal is not to understand every helper on first pass, but to understand what question each file answers.

Recommended route. First read README.md and docs/ARCHITECTURE.md. Then read include/util.h, include/pe.h, include/unwind.h, include/heuristics.h, include/dynamic.h, include/report.h. Only after that read src/main.c, because main.c shows real order of calls. Then read src/pe.c and src/unwind.c slowly. Then heuristics.c. Then report.c. Finally dynamic_win.c, because it is technically hardest and Windows-specific.

1. Read header before implementation: know the contract before details.
2. For each function, ask: what input does it receive, what output does it write, what errors can it return?
3. For each struct, ask: which module fills it and which module consumes it?
4. Do not memorize every static helper. Static helper in C means private function visible only inside that .c file.
5. Mark ownership points: where memory is allocated and where freed.
6. When seeing a term in report, find it in header first, then implementation.
7. Always distinguish file bytes, RVA, file offset and runtime VA.

25. Глоссарий англоязычных терминов проекта

Term	Meaning in project
Analyzer	Инструмент анализа, который не только читает поля, но строит model, checks rules, creates findings and reports.
Parser	Слой чтения формата. Parser extracts fields safely, but by itself does not judge evidence.
Decoder	Слой интерпретации encoded fields. unwind.c decodes UNWIND_INFO and UWOP slots.
Metadata	Служебные данные о коде. EH/Unwind metadata describes functions, stack frame and handlers.
Evidence	Проверяемое наблюдение: конкретный RVA, count, section permissions, parse error, event.
Finding	Structured result of heuristic rule. It is not free text, but data with code/family/severity.
Severity	Серьёзность finding. How important or risky the issue is.
Confidence	Уверенность in interpretation. It is not probability and not severity.
Score	Engineering suspiciousness score 0..100. Transparent triage aid, not ML probability.
Mitigation	Weakening rule that reduces confidence due to benign explanation or insufficient context.
Runtime	State during execution. Dynamic layer observes runtime events.
Static	Analysis without execution; reads file on disk.
Dynamic	Execution-based validation under debugger; Windows-only in this project.
Handler	Code that handles exception/unwind event. UNWIND_INFO may reference handler RVA.
RVA	Relative Virtual Address: address relative to ImageBase.
VA	Virtual Address: actual address in process memory.
file offset	Position of bytes in file on disk.
Section	PE region with virtual/raw ranges and permissions.
Provenance	Information about origin of report: version, timestamp, hash, command line, build.
Schema	Formal structure of JSON report. Defines required fields and types.
CLI	Command-line interface; user controls analyzer by terminal options.
API	Application Programming Interface; functions exposed by module or OS.
Debug API	Windows API that lets debugger observe process events.
Breakpoint	Stop point inserted into target execution for observation.
Fuzzing	Testing with many mutated inputs to find crashes and parser weaknesses.

26. Ответы для защиты именно по устройству проекта

Вопрос комиссии	Ответ
Почему проект не в одном файле?	Потому что разные слои имеют разные responsibilities: safe bytes, PE map, unwind decode, heuristic evidence, dynamic runtime, reports. Разделение снижает ошибки and simplifies testing.
Зачем .h файлы?	Они задают contracts. По ним видно, какие структуры and функции доступны между модулями.
Почему C безопасен для такого проекта?	C сам по себе требует дисциплины, но проект использует bounded reads, fixed-size integers, ownership rules, strict warnings, tests, fuzzing and sanitizers.
Где строится карта?	PE-map строится в pe.c, EH/Unwind-map строится в unwind.c, findings overlay формируется в heuristics.c, отчётная карта выводится in report.c.
Где динамика?	dynamic_win.c, only Windows. It observes runtime events through Windows Debug API and stores DynamicReport.
Где доказательство признаков?	heuristics.c implements rules; docs/evidence_matrix.md documents weights, families, confidence and weakening rules; report.c shows evidence chain.
Где backend/frontend should connect?	Through JSON output. Backend can call CLI and store JSON; frontend displays backend data. Core does not need to become web server.

27. Честные границы реализации

A good defense includes limitations. Project supports PE32+ AMD64 EH/Unwind metadata. It does not claim universal support for x86, ARM64, ELF, Mach-O or managed bytecode. Static layer does not execute target. Dynamic layer is Windows-only and must run untrusted targets in isolated VM.

Project decodes UNWIND_INFO and derives stack/register effects from metadata, but it does not yet fully disassemble function prologues and prove instruction-by-instruction match between UWOP and .text. This would require disassembly layer and architecture semantic matching. That is a realistic future improvement, not a defect that invalidates current structural scope.

Project does not universally parse language-specific handler data after handler RVA. That data depends on handler/runtime, such as __C_specific_handler or C++ EH runtime. Current implementation responsibly records handler RVA and structural context without pretending to know every handler-specific format.

28. Финальная модель, которую нужно держать в голове

Think of project as a chain of models. Buf is bytes. PeImage is PE coordinate map. RuntimeFunction[] is function range map. UnwindSummary[] is unwind metadata map. HeuristicReport is evidence overlay. DynamicReport is runtime evidence overlay. Report output is the artifact read by a human, backend, database, frontend or Ghidra.

If you can explain this chain, you understand project architecture even if you cannot write all C code from memory. For defense, it is enough to know what each module owns, what it receives, what it produces, what it refuses to do and what risks it controls.

The strongest final phrase: EH/Unwind Analyzer is a modular C17 analysis system. Its parser layer safely reads bytes; its PE layer builds address map; its unwind layer decodes Windows x64 EH/Unwind structures; its heuristic layer produces explainable findings; its dynamic layer gathers Windows runtime evidence; its reporting layer turns everything into a verifiable map for reverse engineers and future backend/frontend integration.

29. Приложение: компактная шпаргалка по слоям

Question	File to inspect first
Как файл попадает в память?	util.c, buf_init_from_file
Где проверяется PE32+ AMD64?	pe.c, pe_parse
Где переводится RVA to file offset?	pe.c, pe_rva_to_fileoff
Где читается .pdata?	unwind.c, unwind_read_runtime_functions_ex
Где учитываются UWOP extra slots and padding?	unwind.c, unwind_parse
Где обнаруживается chain loop?	unwind.c, follow_chain block
Где считается score?	heuristics.c, heuristics_analyze/finalize_verdict
Где documented weights?	docs/evidence_matrix.md
Где запускается dynamic?	dynamic_win.c, dynamic_collect_windows
Где пишется JSON?	report.c, report_write_json_ex
Где пишется HTML?	report.c, report_write_html
Где задаётся schema?	docs/report.schema.json
Где подключается Ghidra?	ghidra/import_unwind_csv.py and --ghidra-csv output
Где tests registered?	CMakeLists.txt

30. Источники внутри проекта

- README.md: назначение, static/dynamic features, usage and verification profiles.
- docs/ARCHITECTURE.md: official data flow, modules, ownership, trust boundaries and extension points.
- CMakeLists.txt: build graph, C17, ehua_core, executable, tests and instrumentation options.
- include/*.h: contracts and data structures of all modules.
- src/*.c: current implementation of core, CLI, dynamic collector and reports.
- docs/evidence_matrix.md: 27 findings, evidence families, score, confidence, mitigation and case rules.
- docs/report.schema.json: machine-readable JSON report contract version 1.2.0.
- docs/windows_dynamic_validation.md: Windows dynamic validation checklist and evidence categories.
- tests/: unit, regression, semantic validation, dynamic validation and fuzzing infrastructure.
- ghidra/import_unwind_csv.py: integration with Ghidra workflow.

ПОДРОБНЫЙ ПОФАЙЛОВЫЙ РАЗБОР

Дополнительная учебная часть: функции, внутренние блоки, C-синтаксис и смысл проектных решений без предположения, что читатель уже знает язык C.

A. Как понимать строку C-кода в этом проекте

Чтобы новичок не терялся, нужно научиться читать строку C как грамматическую конструкцию. Почти каждая строка отвечает на один из вопросов: объявляется ли переменная, вызывается ли функция, проверяется ли условие, выделяется ли память, освобождается ли память, записывается ли поле структуры. В этом проекте много строк вида if (!something) return false; Это defensive style: если условие безопасности не выполнено, функция прекращает работу and сообщает ошибку.

Выражение img->exception_rva означает: img is pointer to PeImage, and exception_rva is field inside that PeImage. Arrow operator -> используется, когда есть pointer to struct. Если бы была сама структура, а не pointer, использовалась бы точка: img.exception_rva. Это важно для чтения почти всех модулей: PeImage* img, DynamicReport* dr, StrBuf* sb appear everywhere.

Выражение &img означает “адрес переменной img”. Оно передаётся функции, которая ожидает pointer. Например, pe_parse(&img, &b, &err) says: “вот адрес PeImage, заполни его; вот адрес Buf, читай из него; вот адрес StrBuf, записывай ошибки”. Выражение *out means “object located at pointer out”. В коде это часто скрыто через out->field.

```
PeImage img;
pe_parse(&img, &b, &err);

&img passes address of local structure.
pe_parse fills img fields.
err receives diagnostic text if something fails.

RuntimeFunction* rf = NULL;
size_t rf_count = 0;
unwind_read_runtime_functions_ex(&b, &img, &rf, &rf_count, &err);

&rf is pointer to pointer: function writes newly allocated array pointer into rf.
&rf_count lets function write number of entries.
main.c later owns free(rf).
```

C notation	Meaning
*	Pointer in declarations; dereference in expressions.
&	Address of object; used to pass object to function expecting pointer.
->	Access struct field through pointer.
.	Access struct field directly.
NULL	No valid address stored in pointer.
calloc	Allocate zeroed heap array.
free	Release heap memory previously allocated.
static function	Private helper visible only inside current .c file.

B. main.c по блокам

main.c нужно читать как сценарий одного запуска. Он начинается не с анализа PE, а с разбора командной строки. Командная строка - это то, что пользователь написал after executable name. parse_args converts strings into Args structure. Args is a configuration object: which file to analyze, whether to dump, whether to run heuristics, where to write JSON/HTML, whether dynamic is enabled, and what timeout to use.

После parse_args main.c reads input file. Это первый момент ownership: uint8_t* owned stores memory allocated by buf_init_from_file. Buf b points to this memory and remembers size. main.c owns owned and must release it. If file cannot be read, program prints error and returns 1.

Next main.c initializes StrBuf err. StrBuf is not automatically ready after declaration; sb_init allocates its internal buffer. This pattern repeats: if a module needs error text, caller prepares StrBuf. When finished, caller calls sb_free.

pe_parse is first real analysis call. If it fails, main.c prints “PE parse error” and frees already created objects. Notice that at this point rf/us/hr/dynamic do not exist yet, so cleanup is shorter. This is why cleanup code changes depending on how far the program progressed.

RuntimeFunction reading is slightly different. If exception directory is malformed, main.c may continue with partial structural report. It marks img.exception_directory_oob = true, prints warning, sets exit_code = 1, but keeps going. This is product behavior: malformed input should still produce as much evidence as possible.

The loop for unwind_parse is important. Even if a particular UNWIND_INFO is malformed, the project does not stop entire analysis. us_ok[i] stores success for each entry. This allows later layers to distinguish: function exists, but its unwind summary failed. report.c can show parse_error for that entry, and heuristics.c can count unwind_parse_fail.

When writing JSON/HTML, main.c computes input_display, invocation, timestamp and input_sha256. This is report provenance. It is prepared only when JSON or HTML output is requested, because console-only run does not need full metadata.

1. parse_args
2. buf_init_from_file
3. pe_parse
4. unwind_read_runtime_functions_ex
5. unwind_parse loop
6. heuristics_analyze
7. dynamic_collect_windows
8. metadata preparation
9. report_write_*
10. cleanup and exit code

C. util.c function by function

buf_init_from_file reads file into memory. It opens file, determines size, allocates memory, reads bytes and fills Buf. The function returns bool because I/O can fail: file does not exist, permission denied, allocation failed, short read. The caller receives owned_mem pointer and must free it through buf_free_owned.

buf_read is the core safety function. It does not trust offset and length. It checks whether requested range fits into Buf. If range is valid, it copies bytes to out. If not, it returns false. All typed reads depend on this.

buf_read_u16/u32/u64 read little-endian values. The PE format stores integers little-endian, so reading raw bytes into host integer without explicit conversion would be less clear and less portable. The helper functions make intent visible.

sb_reserve grows StrBuf capacity. It must avoid integer overflow: if new capacity calculation overflows, program could allocate too little and later overwrite memory. The presence of reserve logic shows that report building was considered as a memory-safety problem, not just string concatenation.

sb_appendf is formatted append. It is similar to printf, but appends into StrBuf. It must calculate required length, grow buffer and preserve failed flag. The EHUA_PRINTF_FORMAT macro lets GCC/Clang check format strings at compile time. This is a subtle but professional C detail: compiler can catch wrong printf formats.

file_write_atomic writes output safely. It creates temporary file near destination, writes full content, closes it and replaces final path. This prevents broken report files if program fails in the middle.

sha256_hex implements SHA-256 and returns lowercase hex string. Hex string has 64 characters because SHA-256 digest has 32 bytes and each byte becomes two hex characters. This becomes input_sha256 in reports.

D. pe.c function by function

read_dos_lfanew is private helper. Private means static function visible only inside pe.c. It reads DOS magic and e_lfanew. e_lfanew must point to NT headers. If e_lfanew is outside file, parse cannot continue.

safe_add_u32 prevents unsigned integer overflow. Overflow means arithmetic result does not fit into target type and wraps around. For addresses and offsets this is dangerous: raw_ptr + delta could become small number and pass bounds checks incorrectly. safe_add_u32 returns false if addition would overflow.

ranges_overlap_u64 checks whether two ranges overlap. Range overlap matters for sections: if raw or virtual ranges overlap, mapping becomes ambiguous. The function uses 64-bit arithmetic to avoid overflow when computing ends.

pe_parse is the main public function. It performs a sequence of checks. The order matters. It cannot read section headers before verifying optional header size. It cannot trust DataDirectory before ensuring number_of_rva_and_sizes is large enough. It cannot trust section count before ensuring section headers fit into file.

pe_find_section_for_rva is a lookup helper. It iterates through sections and returns section whose virtual range covers the RVA. It uses max(VirtualSize, RawSize) because some PE files have differences between declared virtual size and raw size. This is a practical parser decision.

pe_rva_to_fileoff converts logical image address to physical file offset. It also rejects RVA in virtual tail: memory may contain zero-filled bytes after loading, but static file parser cannot read bytes that are not present on disk.

E. unwind.c function by function

unwind_reg_name converts register number to human-readable name such as RAX, RBX, RSP or R15. This matters for reports and --ops output.

unwind_flags_to_string converts flags bitmask into readable form. If flags include EHANDLER, report can show that UNWIND_INFO has exception handler. If CHAININFO, tail is interpreted as chained record.

slots_for_code is one of the most important helpers. It encodes the rule “how many two-byte slots this operation occupies”. This prevents CountOfCodes mistakes. It also handles version 2 epilog descriptors. If unknown opcode appears, parser flags UA_BAD_OPCODE and attempts safe continuation only when possible.

parse_unwindinfo_header maps unwind_rva and reads 4 header bytes. If this fails, the parser records diagnostic for UNWIND_INFO header. This is why report can say failure occurred at header, not somewhere vague.

unwind_record_limit determines how far parser may read. It uses section/file boundaries to avoid reading beyond the current available area. available_until_limit calculates how many bytes remain. Together these helpers allow precise expected_size / available_size diagnostics.

unwind_diag_set and unwind_diag_capture_bytes fill diagnostic fields in UnwindSummary. Diagnostic fields are crucial for professional reports: field name, file offset, expected bytes, available bytes, expected slots, available slots and evidence bytes.

parse_chain_rf reads chained RUNTIME_FUNCTION tail. CHAININFO changes tail interpretation: tail is not handler RVA but three DWORDs like BeginAddress, EndAddress, UnwindInfoAddress. If parser ignored flags, it would read wrong meaning.

unwind_parse is the main decoder. It combines all above. It reads header, validates version/flags/frame fields, reads code slots, handles operation-specific operands, computes stack/register summary, reads tail and optionally follows chains. It returns false on structural parse failure but leaves detailed UnwindSummary.

unwind_read_runtime_functions_ex reads .pdata table. It is deliberately strict about directory size multiple of 12. If size is not multiple of RUNTIME_FUNCTION size, last partial record cannot be safely interpreted.

F. heuristics.c internal blocks

heuristics.c contains helper data structures for counting. Reuse analysis needs to know how often a particular RVA appears. This is why the file defines internal hash map logic. The map is not a database; it is temporary memory used during one analysis.

The first analysis phase is counting. It does not immediately create every finding. It scans entries and accumulates facts: invalid ranges, out-of-bounds addresses, unsorted entries, parse failures, anomalies, placement counters, handler counts, reuse counters. Counting first helps make percentage-based decisions later.

The second phase sorts copies of function ranges to check overlaps and duplicate BeginAddress. Sorting is important because input table can be unsorted. If we only checked adjacent entries in original order, overlaps might be missed.

The third phase computes ratios: unwind_reuse_ratio, unwind_unique_ratio, handler_reuse_ratio. Ratio means доля. For example, if max_count is 300 and total is 1000, reuse_ratio is 0.3. Ratios are meaningful only with enough sample size, which is why small_sample_size mitigation exists.

The fourth phase adds findings. add_finding creates HeuristicFinding and copies code/family/severity/message. annotate_finding attaches representative RVA and affected_count. This separation matters: some findings have location, some are global combinations and may not have one representative address.

The fifth phase applies combinations. Combo finding means multiple signals together produce stronger evidence than each alone. Example: writable unwind metadata plus writable handler target is more suspicious than either alone. placement_integrity_combo links placement issues with parse/integrity issues.

The final phase computes verdict and confidence. finalize_verdict counts weak, medium and strong findings, calculates evidence level, applies bonuses and weakening rules, clamps confidence and sets requires_manual_review. This is the final static assessment.

G. dynamic_win.c internal blocks

dynamic_win.c has two compiled personalities. Under _WIN32 it includes Windows headers and real implementation. Outside _WIN32 it compiles stub functions. Stub means placeholder implementation that returns controlled result. This lets static analyzer build on macOS/Linux without pretending dynamic works there.

ModList stores loaded modules. Module in Windows process is executable image loaded in memory: main EXE or DLL. The collector refreshes module list because API breakpoints can only be set after relevant DLLs are loaded.

BpList stores breakpoints. Each breakpoint has address, original byte, armed state and kind bits. Kind bits allow one address to represent multiple roles. For example, a breakpoint can be API breakpoint or custom handler breakpoint.

build_cmdline is careful because Windows command line quoting rules are tricky. Arguments with spaces or backslashes must be quoted correctly. This prevents target launch from receiving wrong arguments.

local_proc_loc finds local address of API symbol in analyzer process, then dynamic code maps it to target module base. This is a common debugger technique: system DLL layout can be related by module base plus offset, but the code still tracks modules to avoid blind assumptions.

Debug loop is the heart. It repeatedly calls WaitForDebugEvent. For CREATE_PROCESS_DEBUG_EVENT, it stores base image and sets handler/API breakpoints. For LOAD_DLL_DEBUG_EVENT, it refreshes modules and sets API breakpoints if newly available. For EXCEPTION_DEBUG_EVENT, it distinguishes normal exceptions from breakpoint/single-step events. For EXIT_PROCESS_DEBUG_EVENT, it stores exit code and finishes.

When breakpoint fires, RIP points after INT3, so code rewinds RIP to breakpoint address, clears breakpoint, sets Trap Flag for single-step, captures API arguments from registers RCX, RDX, R8, R9 on Windows x64 calling convention, records event and re-arms breakpoint after single-step. This is advanced but important: it means dynamic collector observes without permanently replacing target instructions.

dynamic_win.c is the only layer that executes target. Static reports can be generated safely on unknown files; dynamic runs need isolated Windows VM for untrusted samples.

H. report.c internal blocks

report.c begins with helpers for formatting register masks, anomaly bitmasks, section names and permissions. These helpers translate compact internal bitmasks into readable text. For example, saved_nonvol_mask must become RBX/RBP/RDI-like list for the user.

finding_rule_text, finding_expected_text, finding_impact_text and Russian explanation helpers are part of interpretability. Interpretability means user can understand why tool made a statement. A report that says only “rf_overlaps” requires user to know code. A report that explains overlapping RUNTIME_FUNCTION ranges create ambiguous ControlPc mapping is much stronger.

CSV writer emits tabular rows. In normal mode it exports function/unwind table. In Ghidra mode it exports a format suitable for importer. csv_cell handles escaping because CSV cells may contain quotes, commas or line breaks.

JSON writer builds strict object matching schema. It includes input, report_metadata, image_base, sections, heuristic_analysis, dynamic_validation, assessment and functions. It writes hex values as strings like 0x00001234, which avoids ambiguity with signed/unsigned integer display and preserves address-like formatting.

HTML writer builds self-contained dashboard. It includes CSS inside document and Content Security Policy. The policy blocks network assets. This matters because report may be opened on analyst machine and should not load remote scripts.

Key diagnostics panel lifts important parse failures to top-level. Without it, user would have to scroll through large function table. Evidence block explains each finding as chain: PE -> Exception Directory -> RuntimeFunction -> UnwindInfoAddress -> section/file offset -> diagnostic. This is the report manifestation of “project builds map”.

I. docs and tests as code-adjacent architecture

docs/evidence_matrix.md is not decorative documentation. It freezes the engineering evidence matrix: finding codes, families, severity rules, score weights, confidence model, weakening rules and final case rules. Without this document, heuristics.c would look arbitrary. With it, the project can answer “why this score” and “why this finding severity”.

docs/report.schema.json is also part of architecture. Schema forces JSON reports to remain stable. If backend/frontend/database later consume reports, schema prevents accidental field changes from silently breaking integration. Tests can validate reports against schema.

docs/windows_dynamic_validation.md is the operational contract for dynamic. It states how to build on Windows, which smoke scenarios must pass and what evidence is expected. This matters because dynamic is platform-specific and must be validated on Windows x64, not only on macOS.

tests/ is a validation layer. It contains C unit tests, Python validation scripts, sample builders, fuzzing entry point and dynamic smoke scripts. For a C tool parsing untrusted binary data, tests are not “bonus”; they are part of trust story. They show how assumptions are checked.

J. Расширенный маршрут чтения проекта за несколько дней

Day 1 should be architecture and C basics. Read README.md, docs/ARCHITECTURE.md and this document. Then read include/*.h only. Do not enter long .c files yet. Your goal is to recognize names: Buf, PeImage, RuntimeFunction, UnwindSummary, HeuristicReport, DynamicReport, ReportMetadata.

Day 2 should be main.c and util.c. Read parse_args lightly, then main function carefully. Draw arrows between created objects. Then read util.c and understand safe reads and StrBuf. At this stage you should be able to explain how file bytes enter the program and who frees memory.

Day 3 should be pe.c. Read pe_parse and write down each PE field: MZ, e_lfanew, PE signature, COFF header, optional header, ImageBase, alignments, sections, Exception Directory. Then read pe_rva_to_fileoff until you can explain RVA vs file offset without confusion.

Day 4 should be unwind.c. Read RuntimeFunction first, then UNWIND_INFO header, then slots, then padding, then handlers and chain. Do not try to memorize all UWOP details at once. Understand the rule: parser must move by slots, not by visual operations.

Day 5 should be heuristics.c and docs/evidence_matrix.md. Read families first, then findings. Understand score/confidence/severity separation. Be able to explain why tool is not ML and not malware classifier.

Day 6 should be report.c. Focus on JSON/HTML structure, evidence chain and static heuristic matrix. You do not need to memorize every CSS string. Understand why report.c is large and why output layer is part of product.

Day 7 should be dynamic_win.c. Read dynamic.h first. Then understand Debug API loop conceptually: create process, wait events, set breakpoints, capture events, continue process. Do not start from every Windows HANDLE detail. The defense-level understanding is runtime evidence collection and limitations.

K. Final defense narrative, expanded

When presenting architecture, begin with purpose. “The project analyzes Windows x64 EH/Unwind metadata in PE32+ AMD64 files. It builds a structural map of exception/unwind data and reports anomalies for reverse engineers.” Then immediately show pipeline. This prevents commission from reducing project to one parser function.

Then explain C design. “The code is separated into headers and implementations. Headers define contracts and data models. Implementations fill or consume those models. The core is built as ehua_core library, and main.c is a CLI orchestrator.” This shows engineering maturity.

Then walk layer by layer. “util.c makes byte reading safe. pe.c builds PE coordinate map. unwind.c decodes .pdata/.xdata. heuristics.c turns decoded facts into findings. dynamic_win.c optionally observes runtime events on Windows. report.c serializes everything into JSON/HTML/CSV and explains findings.” This is the strongest compact explanation.

Then name constraints honestly. “Static analysis does not execute target. Dynamic mode executes target and must be isolated for untrusted samples. The project does not claim malware classification and does not yet perform full disassembly-based prolog matching. Its scope is structural EH/Unwind analysis with evidence-based reporting.” Honesty protects the project from overclaiming.

Главная цель этого приложения - чтобы ты мог смотреть на любой файл проекта и задавать правильные вопросы: что этот файл получает, какую модель строит, какие ошибки ловит, что отдаёт дальше, кто владеет памятью and как это появляется в отчёте.